CSA1220 – Computer Architecture **R Hardhik**
CO2 – AT2 **192312417**

Q1. Debugging Signed Integer Addition (2's Complement Overflow)

Solution

Given numbers:

- +120 = **01111000_2**
- +50 = **00110010_2**

Addition:

  01111000   (+120)

+ 00110010   (+50)

  10101010

Result = **10101010_2**

In 2's complement, this represents:

Invert : 01010101

+1   01010110 = 86

So the result is **-86**, which is incorrect because:

120 + 50 = 170

The maximum value representable in an 8-bit signed integer is **+127**. Since **170 > 127**, **overflow occurs**.

Overflow Detection

Overflow occurs when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

It can also be detected by checking:

Carry into MSB ≠ Carry out of MSB

Solution

- Detect overflow using the overflow flag (V).
- Increase data width (e.g., 16-bit integers).
- Generate an exception or error message when overflow occurs.

Theory

In 2's complement representation, signed numbers have a limited range.

For 8 bits:

- Minimum = **−128**
- Maximum = **+127**

When an arithmetic result exceeds this range, overflow occurs, causing an incorrect signed result. Modern processors detect overflow using hardware logic and set an overflow flag so software can handle the error.

Q2. Ripple Carry Adder Delay and Carry Look-Ahead Adder

Solution

In a Ripple Carry Adder (RCA), each full adder waits for the carry from the previous stage.

Example:

Bit0 → Carry0

↓

Bit1 → Carry1

↓

Bit2 → Carry2

↓

Bit3 → Carry3

The carry ripples through every stage.

For an n-bit adder:

Delay ∝ Number of bits

This creates a performance bottleneck in real-time systems.

Optimized Solution: Carry Look-Ahead Adder (CLA)

CLA calculates carries simultaneously using:

Generate (G) = A × B

Propagate (P) = A ⊕ B

Carry equations:

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

...

All carries are generated in parallel, greatly reducing delay.

Theory

A Ripple Carry Adder is simple and requires less hardware, but its delay increases with the number of bits because carries propagate sequentially.

A Carry Look-Ahead Adder predicts carry values using generate and propagate signals, allowing parallel carry computation. This significantly improves speed and is widely used in high-performance processors.

Q3. Debugging Booth's Multiplication Algorithm

Solution

Possible reasons for incorrect multiplication of negative numbers:

1. Incorrect bit-pair checking (Q_0 and Q_{-1}).
2. Wrong arithmetic right shift.
3. Missing sign extension.
4. Incorrect initialization of Q_{-1}.

Booth's decision table:

Q_0 Q_{-1}	Operation
00	No operation
01	A = A + M
10	A = A − M
11	No operation

After each operation:

- Perform **Arithmetic Right Shift** on **A, Q, Q_{-1}**
- Preserve the sign bit.

Corrections

- Correctly initialize $Q_{-1} = 0$.
- Preserve MSB during arithmetic shift.
- Extend sign bits properly.
- Verify subtraction uses 2's complement.

Theory

Booth's algorithm efficiently multiplies signed binary numbers by examining pairs of bits. It reduces the number of additions and subtractions for consecutive 1's.

Accurate multiplication depends on correct bit-pair evaluation, arithmetic right shifting with sign extension, and proper initialization. Errors in these steps mainly affect negative operands.

Q4. Restoring vs Non-Restoring Division

Solution

In Restoring Division:

1. Shift dividend.
2. Subtract divisor.
3. If result becomes negative:
   - Restore previous value by adding divisor again.
4. Continue.

Problem:

Many iterations require an additional restoration step.

Subtract

↓

Negative?

↓ Yes

Restore

↓

Next iteration

This increases execution time.

Optimized Solution: Non-Restoring Division

Instead of restoring immediately:

- If result is negative, perform addition in the next iteration.
- If result is positive, continue subtraction.

This eliminates repeated restoration operations.

Advantages:

- Fewer arithmetic operations.
- Faster execution.
- Better processor efficiency.

Theory

Restoring Division restores the partial remainder whenever subtraction produces a negative value, increasing hardware operations and delay.

Non-Restoring Division avoids immediate restoration by adjusting the next operation based on the sign of the previous remainder. This reduces execution time and improves performance in embedded systems.

Q5. IEEE 754 Floating Point Accuracy

Solution

Problem:

Adding a very large number and a very small number.

Example:

1.0×10^8

+

1.0

Steps:

1. **Exponent Alignment**

The smaller number's mantissa is shifted right repeatedly to match the larger exponent.

2. **Loss of Precision**

After many shifts:

000000...0001

The significant bits may disappear.

3. **Addition**

Only the large number remains significant.

4. **Normalization**

Result is normalized.

5. **Rounding**

Remaining bits are rounded according to IEEE 754 rules, introducing small errors.

Optimization

- Use **double precision (64-bit)** instead of single precision.
- Use compensated summation algorithms (e.g., **Kahan Summation**).
- Add numbers of similar magnitude first.
- Avoid repeated accumulation of very small values into very large values.

Theory

IEEE 754 single precision uses:

- **1-bit Sign**
- **8-bit Exponent**
- **23-bit Mantissa**

During floating-point addition, exponents must first be aligned. Large exponent differences force the smaller number to lose significant bits, causing rounding errors and reduced accuracy.

Using double precision or improved summation algorithms increases precision and minimizes numerical errors, making computations more reliable in scientific applications.